

Lecture 8

Homomorphic Encryption: Definitions and First Examples

SEC 595: Encrypted Computing

Roadmap

Lecture 7 built a scheme that encrypts. This lecture asks what it takes to *compute* on ciphertexts. Section 1 defines a homomorphic encryption scheme and the properties it must have: correctness, compactness and security, and the classification PHE / SWHE / FHE. Section 2 gives three partially homomorphic schemes — RSA, ElGamal and Paillier — each with a worked instance. Section 3 gives a small scheme that supports both addition and multiplication, and shows where its limit comes from.

1. Homomorphic Encryption

1.1 The Four Algorithms

Definition 1 (Homomorphic encryption scheme). A homomorphic encryption (HE) scheme consists of four algorithms.

- $\text{KeyGen}(\lambda) \rightarrow (\text{Pk}, \text{Sk})$: given the security parameter λ , output a public key and a secret key.
- $\text{Enc}(\text{Pk}, \text{pt}) \rightarrow \text{ct}$: encrypt a plaintext.
- $\text{Dec}(\text{Sk}, \text{ct}) \rightarrow \text{pt}$: decrypt a ciphertext.
- $\text{Evaluate}(\text{Pk}, \Pi, \text{ct}_1, \dots, \text{ct}_k) \rightarrow \widehat{\text{ct}}$: apply a function Π to ciphertexts, producing a ciphertext.

The first three are ordinary public-key encryption; Regev's scheme of Lecture 7 has all three. The fourth is new, and its signature is the point: Evaluate takes neither the secret key nor the plaintexts. It can be run by a server that holds neither.

The function Π is an input to Evaluate, not part of the scheme. A scheme is homomorphic *for* a class of functions — addition, multiplication, or whole circuits — and that class must always be named.

1.2 Correctness

Definition 2 (Correctness). An HE scheme is correct for Π if, for all plaintexts,

1. $\text{Dec}(\text{Sk}, \text{Enc}(\text{Pk}, \text{pt})) = \text{pt}$, and
2. $\text{Dec}(\text{Sk}, \text{Evaluate}(\text{Pk}, \Pi, \text{ct}_1, \dots, \text{ct}_k)) = \Pi(\text{pt}_1, \dots, \text{pt}_k)$, where $\text{ct}_i = \text{Enc}(\text{Pk}, \text{pt}_i)$.

Property 1 says the scheme is an encryption scheme. Property 2 is the homomorphism: computing on ciphertexts and then decrypting agrees with decrypting and then computing. For lattice schemes both hold with probability $1 - \text{negl}(\lambda)$ rather than 1, because the error has unbounded support (Lecture 7).

1.3 The Commuting Square

Property 2 is usually drawn as a square:

$$\begin{array}{ccc}
 \boxed{\text{pt}_1, \text{pt}_2} & \xrightarrow{\text{Enc}} & \boxed{\text{ct}_1, \text{ct}_2} \\
 \downarrow +, \times & & \downarrow \boxplus, \boxtimes \\
 \boxed{\text{pt}_1 * \text{pt}_2} & \xleftarrow{\text{Dec}} & \boxed{\text{ct}_1 \hat{*} \text{ct}_2}
 \end{array}$$

The operations $\boxplus$ and $\boxtimes$ on ciphertexts are whatever the scheme supplies; they are rarely the same as $+$ and $\times$. In Paillier below, $\boxplus$ is multiplication of ciphertexts.

Definition 3 (Homomorphic property). Enc is homomorphic with respect to $*$ on plaintexts and $\hat{*}$ on ciphertexts if

$$\text{Dec}(\text{Sk}, \text{Enc}(\text{pt}_1) \hat{*} \text{Enc}(\text{pt}_2)) = \text{pt}_1 * \text{pt}_2.$$

For a deterministic Enc this can be written as the equality of ciphertexts $\text{Enc}(\text{pt}_1) \hat{*} \text{Enc}(\text{pt}_2) = \text{Enc}(\text{pt}_1 * \text{pt}_2)$; for a randomised one the equality holds only after decryption.

1.4 Compactness

Correctness alone is too weak. The “scheme” whose Evaluate returns $(\Pi, \text{ct}_1, \dots, \text{ct}_k)$ unchanged, and whose Dec decrypts the inputs and then applies Π , satisfies both properties — and the server has computed nothing.

Definition 4 (Compactness). An HE scheme is **compact** if the size of every output of Evaluate is bounded by a polynomial in λ , independent of Π and of the number of inputs.

Equivalently, the cost of Dec does not grow with the work Evaluate did. Compactness concerns size, not speed: Evaluate may be slow, but what it returns is no larger than a fresh ciphertext. Every scheme in this course is compact.

1.5 Security

Definition 5 (IND-CPA security). The challenger runs KeyGen and gives Pk to the attacker. The attacker chooses two plaintexts m_0, m_1 ; the challenger picks a random bit b and returns $\text{ct}^* = \text{Enc}(\text{Pk}, m_b)$; the attacker outputs a guess b' . The scheme is IND-CPA secure if, for every efficient attacker,

$$\text{Adv} = \left| \Pr[b' = b] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

IND-CPA is also called **semantic security**: the ciphertext reveals nothing about the plaintext, not even which of two known candidates it is.

Two consequences matter for HE.

1. *Encryption must be randomised.* If Enc is deterministic, the attacker computes $\text{Enc}(\text{Pk}, m_0)$ itself and compares it with ct^* , winning with probability 1.
2. *HE cannot be CCA2 secure.* In the chosen-ciphertext game the attacker may ask for decryptions of any ciphertext except ct^* . With Evaluate it forms a different ciphertext of a related plaintext (for instance $\text{ct}^* \boxtimes \text{Enc}(1)$), has it decrypted, and reads off m_b . Homomorphic schemes therefore aim for IND-CPA.

1.6 Classification

Schemes are classified by *which* operations Evaluate supports and *how many times*.

Definition 6 (PHE, SWHE, LHE, FHE). • **PHE** (partially homomorphic): one operation, addition or multiplication, an unlimited number of times.

- **SWHE** (somewhat homomorphic): both operations, a limited number of times.
- **LHE** (levelled homomorphic): both operations, up to a multiplicative depth L fixed at key generation.
- **FHE** (fully homomorphic): both operations, an unlimited number of times.

Addition and multiplication together suffice for any computation: over $\mathbb{Z}_2$ they are XOR and AND, from which every Boolean circuit is built. So an FHE scheme can evaluate any function.

	Operations	How many	Examples
PHE	one	unlimited	RSA, ElGamal, Paillier (Section 2)
SWHE	both	limited	the scheme of Section 3
LHE	both	depth L	BGV, BFV, CKKS as deployed
FHE	both	unlimited	Section 3 with bootstrapping

2. Partially Homomorphic Encryption

2.1 Unpadded RSA

Choose primes p, q , set $n = pq$, choose e coprime to $(p-1)(q-1)$, and let $d = e^{-1} \bmod (p-1)(q-1)$. Then $\text{Pk} = (e, n)$, $\text{Sk} = d$, and

$$\text{Enc}(x) = x^e \bmod n, \quad \text{Dec}(\text{ct}) = \text{ct}^d \bmod n.$$

Proposition 1. *Unpadded RSA is multiplicatively homomorphic:* $\text{Enc}(x) \text{Enc}(y) \equiv x^e y^e = (xy)^e = \text{Enc}(xy) \pmod{n}$.

Example 1 ($n = 143$). Take $p = 11$, $q = 13$, so $n = 143$ and $(p-1)(q-1) = 120$. With $e = 7$, $d = 103$, since $7 \cdot 103 = 721 \equiv 1 \pmod{120}$. Encrypt $x = 5$ and $y = 7$:

$$\text{ct}_1 = 5^7 \bmod 143 = 47, \quad \text{ct}_2 = 7^7 \bmod 143 = 6.$$

Then $\text{ct}_1 \text{ct}_2 = 282 \equiv 139$, and $139^{103} \bmod 143 = 35 = 5 \times 7$. ✓ Addition fails: $\text{ct}_1 + \text{ct}_2 = 53$ decrypts to $53^{103} \bmod 143 = 14 \neq 12$.

There is no ciphertext operation giving $\text{Enc}(x+y)$: $(x+y)^e$ contains the cross terms $x^j y^{e-j}$, which cannot be formed from x^e and y^e without taking e -th roots. RSA is PHE for $\times$.

Remark. Unpadded RSA is *not* IND-CPA secure, because Enc is deterministic (Section 1.5). Deployed RSA pads the message with randomness, which also destroys the homomorphism.

2.2 ElGamal

Definition 7 (ElGamal). Let P be a prime and g a primitive root modulo P .

- KeyGen: choose a random X ; set $y = g^X \bmod P$. $\text{Pk} = (P, g, y)$, $\text{Sk} = X$.
- $\text{Enc}(m)$, $m \in \mathbb{Z}_P^*$: choose a random k and output $(c_1, c_2) = (g^k, m y^k) \bmod P$.
- $\text{Dec}(c_1, c_2)$: compute $s = c_1^X$ and output $m = c_2 s^{-1} \bmod P$.

Decryption works because $s = (g^k)^X = (g^X)^k = y^k$, the mask. Each encryption uses a fresh k , so ElGamal is randomised; it is IND-CPA secure under the DDH assumption (Lecture 1).

Proposition 2. *ElGamal is multiplicatively homomorphic. Componentwise multiplication gives*

$$(g^{k_1}, m_1 y^{k_1}) \cdot (g^{k_2}, m_2 y^{k_2}) = (g^{k_1+k_2}, (m_1 m_2) y^{k_1+k_2}),$$

an encryption of $m_1 m_2$ with randomness $k_1 + k_2$.

Example 2 ($P = 23$, $g = 5$). The powers of 5 modulo 23 run through all of $\{1, \dots, 22\}$, so 5 is a primitive root. With $X = 6$, $y = 5^6 \bmod 23 = 8$.

Encrypt $m = 10$ with $k = 10$: $c_1 = 5^{10} \bmod 23 = 9$ and $c_2 = 10 \cdot 8^{10} = 10 \cdot 3 \equiv 7$. The ciphertext is $(9, 7)$.

Decrypt: $s = 9^6 \bmod 23 = 3$, $s^{-1} = 8$ (since $3 \cdot 8 = 24 \equiv 1$), and $m = 7 \cdot 8 = 56 \equiv 10$. ✓

Multiply: encrypt $m_1 = 3$ with $k_1 = 10$ and $m_2 = 4$ with $k_2 = 3$, giving $(9, 9)$ and $(10, 1)$. Their product is $(90, 9) \equiv (21, 9)$. Decrypting, $s = 21^6 \bmod 23 = 18$, $s^{-1} = 9$, and $m = 9 \cdot 9 = 81 \equiv 12 = 3 \times 4$. ✓

2.3 Paillier

Paillier is additively homomorphic. It works modulo n^2 and places the message in an exponent.

Definition 8 (Paillier). • **KeyGen**: choose primes p, q with $\gcd(pq, (p-1)(q-1)) = 1$. Set $n = pq$, $\lambda = \text{lcm}(p-1, q-1)$, choose $g \in \mathbb{Z}_{n^2}^*$, and set $\mu = L(g^\lambda \bmod n^2)^{-1} \bmod n$, where $L(x) = (x-1)/n$. $\text{Pk} = (n, g)$, $\text{Sk} = (\lambda, \mu)$.

- **Enc**(m), $0 \leq m < n$: choose a random $r \in \mathbb{Z}_n^*$ and output $c = g^m r^n \bmod n^2$.
- **Dec**(c): output $m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n$.

L is integer division; it is applied only to values $\equiv 1 \pmod{n}$. The inverse defining μ must exist, which holds for most but not all g ; the common choice $g = n+1$ always works and gives $\mu = \lambda^{-1} \bmod n$. Decryption is correct because $r^{n\lambda} \equiv 1 \pmod{n^2}$, so raising to λ removes the randomiser, and L then reads off m times a constant that μ cancels. Paillier is IND-CPA secure under the decisional composite residuosity assumption.

Proposition 3. *Paillier is additively homomorphic:*

$$c_1 c_2 = g^{m_1} r_1^n \cdot g^{m_2} r_2^n = g^{m_1+m_2} (r_1 r_2)^n \bmod n^2 = \text{Enc}(m_1 + m_2 \bmod n).$$

Moreover, for a public constant k , c^k decrypts to $km \bmod n$.

Multiplying ciphertexts adds plaintexts: here $\boxplus$ is $\times$. Two encrypted values cannot be multiplied, so Paillier is PHE for $+$.

Example 3 ($p = 7$, $q = 11$). *Keys*. $n = 77$, $n^2 = 5929$, $\gcd(77, 60) = 1$, $\lambda = \text{lcm}(6, 10) = 30$. Take $g = 5$: $5^{30} \bmod 5929 = 1926$, $L(1926) = 1925/77 = 25$, and $\mu = 25^{-1} \bmod 77 = 37$. $\text{Pk} = (77, 5)$, $\text{Sk} = (30, 37)$.

Encrypt $m_1 = 15$ with $r = 23$: $c_1 = 5^{15} \cdot 23^{77} = 1266 \cdot 606 \equiv 2355 \pmod{5929}$.

Decrypt: $2355^{30} \bmod 5929 = 5160$, $L(5160) = 67$, and $m = 67 \cdot 37 \bmod 77 = 15$. ✓

Add: encrypt $m_2 = 20$ with $r = 30$, giving $c_2 = 1388$. Then $c_1 c_2 \bmod 5929 = 1861$, which decrypts to $35 = 15 + 20$. ✓ And c_1^3 decrypts to $45 = 3 \times 15$.

Wraparound: plaintexts live in $\mathbb{Z}_{77}$, so adding encryptions of 70 and 20 gives $90 \equiv 13$.

3. A Scheme With Both Operations

PHE schemes support one operation because their algebra supports one. A scheme for both must add and multiply the same ciphertexts. The simplest example works over the integers (van Dijk, Gentry, Halevi and Vaikuntanathan, 2010) and encrypts one bit at a time.

3.1 The Scheme

Definition 9 (Integer HE scheme). • **KeyGen**: the secret key is a large odd integer p .

- **Enc**(m), $m \in \{0, 1\}$: choose a random large q and a random small r , and output

$$c = pq + 2r + m.$$

- **Dec**(c): compute $c \bmod p$ in the centred range $(-p/2, p/2]$, and output it modulo 2.

Call $2r + m$ the **noise**. Decryption is correct while $|2r + m| < p/2$: then $c \bmod p = 2r + m$ exactly, and its parity is m . (The version above uses p itself to encrypt, so it is a secret-key scheme; a public-key version publishes many encryptions of 0 and adds a random subset of them. Security rests on the *approximate-GCD* problem: given many near multiples $pq_i + 2r_i$ of p , find p .)

3.2 Addition and Multiplication

Proposition 4. Let $c_i = pq_i + 2r_i + m_i$. Then

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2),$$

$$c_1 c_2 = p(\dots) + 2(2r_1 r_2 + r_1 m_2 + r_2 m_1) + m_1 m_2.$$

So $c_1 + c_2$ encrypts $m_1 \oplus m_2$ and $c_1 c_2$ encrypts $m_1 \wedge m_2$, provided the new noise stays below $p/2$.

Both operations are ordinary integer arithmetic. Addition adds the noises; multiplication multiplies them.

Example 4 ($p = 101$). Encrypt $m_1 = 1$ with $q = 7$, $r = 2$, and $m_2 = 1$ with $q = 5$, $r = 3$:

$$c_1 = 707 + 4 + 1 = 712, \quad c_2 = 505 + 6 + 1 = 512.$$

Their noises are $712 \bmod 101 = 5$ and $512 \bmod 101 = 7$; both are odd, so both decrypt to 1.

Add: $c_1 + c_2 = 1224$, and $1224 \bmod 101 = 12$, which is even: $1 \oplus 1 = 0$. ✓

Multiply: $c_1 c_2 = 364544$, and $364544 \bmod 101 = 35 = 5 \times 7$, which is odd: $1 \wedge 1 = 1$. ✓

Multiply again: c_2^2 has noise $7^2 = 49 < 50.5$ and decrypts to 1. But c_2^3 has noise $7^3 = 343 > 50.5$: $343 \bmod 101 = 40$, even, so it decrypts to 0 instead of 1. ✗

3.3 Somewhat, Then Fully

The scheme supports both operations, but only until the noise reaches $p/2$. Since multiplication multiplies noise, the number of multiplications in sequence — the *depth* — is limited. The scheme is SWHE; choosing p large enough for a fixed depth makes it LHE.

Gentry (2009) showed how to remove the limit. If a scheme can evaluate its own decryption circuit homomorphically, with room for one more operation, then a noisy ciphertext can be *refreshed*: encrypt it again and evaluate Dec under encryption, using an encryption of the secret key. The result encrypts the same bit with fresh noise. This is **bootstrapping**, and applied to an SWHE scheme it yields FHE. Lecture 11 develops it.

Summary

1. An HE scheme is KeyGen, Enc, Dec and Evaluate; Evaluate uses neither the secret key nor the plaintexts.
2. It must be correct for Π (decrypting an evaluated ciphertext gives Π of the plaintexts), compact (outputs of fixed size), and IND-CPA secure (hence randomised). Homomorphic schemes cannot be CCA2 secure.
3. PHE supports one operation without limit; SWHE both, with a limit; LHE both, to a fixed depth; FHE both, without limit.
4. RSA and ElGamal are multiplicative PHE; Paillier is additive PHE, with multiplication of ciphertexts acting as addition of plaintexts.
5. The integer scheme $c = pq + 2r + m$ adds and multiplies bits, but multiplication multiplies noise, so its depth is limited. Bootstrapping removes the limit.

Exercises

1. With the RSA keys of Example ??, encrypt $x = 3$ and $y = 4$, multiply the ciphertexts, and decrypt. Then show that unpadded RSA fails IND-CPA by describing the attacker explicitly.
2. In Example ??, encrypt $m = 2$ with $k = 5$ and $m = 6$ with $k = 7$. Multiply the ciphertexts, verify that the product has the form $(g^{12}, 12y^{12})$, and decrypt.
3. In Example ??, compute $c_1^2 c_2$ and predict its decryption before computing it.
4. Given a Paillier ciphertext c of an unknown m , construct a ciphertext of $m + 1$ without the secret key. Use this to win the CCA2 game against Paillier.
5. Repeat the Paillier key generation of Example ?? with $g = n + 1 = 78$. Verify $L(g^\lambda \bmod n^2) = \lambda$ and find μ .
6. In the integer scheme with $p = 101$, encrypt $m = 0$ with $q = 3$, $r = 4$. Compute the noise after one and after two multiplications by $c_2 = 512$. Which results decrypt correctly?
7. In the integer scheme, fresh noise is at most B . Show that a product of k fresh ciphertexts has noise at most roughly B^k , and find the largest k that decrypts correctly when $p \approx 2^{100}$ and $B = 2^{10}$.